
CPU SCHEDULING SIMULATOR ANALYSIS

SJF vs Priority Comparison Project

Course	Operating Systems
Repository URL	https://github.com/razan-106/SJF-vs-Priority-Comparison-Project
Submission Date	10/5/2026

1. Project Overview

This project presents a comprehensive CPU Scheduling Simulator developed as a practical investigation into two of the most widely studied scheduling paradigms in operating systems literature: Shortest Job First (SJF) and Priority Scheduling. By implementing both algorithms in preemptive and non-preemptive variants, the simulator facilitates a structured, empirical comparison across four distinct scheduling configurations.

The central research question addressed by this project revolves around the fundamental trade-off between efficiency and urgency in process scheduling. SJF-based algorithms optimise average waiting time by favouring processes with shorter burst durations; however, they may deprive high-priority but long-running processes of CPU time. Conversely, Priority Scheduling ensures that critical or urgent processes receive immediate attention, yet may introduce inefficiencies in overall throughput if not balanced appropriately.

The four scheduling variants implemented and evaluated are as follows:

- Non-Preemptive SJF: The process with the shortest burst time among all currently available processes is selected and executed to completion before any rescheduling occurs.
- Preemptive SJF: At each millisecond interval, the scheduler re-evaluates the ready queue and preempts the currently running process if a newly arrived process has a shorter remaining burst time.
- Non-Preemptive Priority Scheduling: The process with the highest priority (lowest numerical value) among available processes is selected and runs without interruption until completion.
- Preemptive Priority Scheduling: The scheduler continuously evaluates priorities, preempting the running process whenever a higher-priority process becomes available.

Through controlled test scenarios and quantitative performance metrics — including Average Waiting Time (WT), Average Turnaround Time (TAT), and CPU utilisation — this simulator enables students and practitioners to develop an intuitive and data-driven understanding of scheduling algorithm behaviour under diverse workload conditions.

2. Technical Stack

The simulator is implemented entirely in Python, leveraging a curated selection of modern libraries to deliver both computational correctness and an interactive user experience. The following table summarises the core components of the technical stack:

Component	Library / Tool	Role in Simulator
Programming Language	Python 3.10+	Core scheduling logic, data processing, and backend computation
Web Framework	Streamlit	Interactive browser-based UI; real-time input/output rendering
Data Manipulation	Pandas	Tabular management of process attributes, results, and metrics
Data Visualisation	Plotly	Dynamic Gantt charts and comparative bar graphs for scheduling output

Python was selected for its expressiveness and extensive ecosystem, making it well-suited for algorithm simulation. Streamlit provides a declarative, widget-based approach to UI development, enabling rapid iteration without front-end boilerplate. Pandas structures process data into DataFrames that seamlessly feed Plotly's visualisation engine, producing publication-quality Gantt charts and metric comparisons.

3. Implementation Logic

3.1 Simulation Time Model

The simulator adopts a discrete-event model operating at 1-millisecond (1 ms) granularity. At each unit of simulated time, the scheduler engine performs the following sequence of operations:

- Advance the global clock by 1 ms.
- Admit all processes whose arrival time equals the current clock value into the ready queue.
- Invoke the scheduling policy to determine which process — if any — should occupy the CPU.
- Decrement the remaining burst time of the executing process by 1 ms.
- Record the CPU state for Gantt chart construction.
- Check for process completion; compute Waiting Time and Turnaround Time upon termination.

This 1 ms evaluation granularity is critical for preemptive variants, as it ensures that preemption opportunities are evaluated at every possible point, faithfully replicating the behaviour of a real-time OS scheduler.

3.2 Preemption Mechanism

In preemptive scheduling modes (Preemptive SJF and Preemptive Priority), the scheduler re-evaluates at each millisecond whether the currently executing process should be displaced. For Preemptive SJF (SRTF), this comparison is based on remaining burst time; for Preemptive Priority, it is based on the numerical priority value (lower value indicates higher urgency).

When a preemption event is triggered, the current process's remaining time is preserved in the process control block (PCB), and it is returned to the ready queue. Context-switch overhead is modelled as negligible in this academic implementation.

3.3 Tie-Breaking via FCFS

Tie-breaking is a critical correctness consideration in all four algorithms. Whenever two or more processes share an identical scheduling key (equal burst time for SJF, or equal priority for Priority Scheduling), the simulator resolves the tie using First-Come, First-Served (FCFS) ordering — i.e., the process with the earlier arrival time is selected. If arrival times are also equal, the process with the lower process ID is preferred. This deterministic tie-breaking ensures reproducible and unambiguous simulation results.

3.4 Performance Metrics

For each process p , the simulator computes:

- Waiting Time (WT): The total duration that process p resides in the ready queue without executing. $WT = \text{Completion Time} - \text{Arrival Time} - \text{Burst Time}$.
- Turnaround Time (TAT): The total elapsed time from process arrival to completion. $TAT = \text{Completion Time} - \text{Arrival Time}$.

- Average WT and Average TAT are computed across all processes and presented as the primary comparative metrics between algorithms.

4. Interface Sections

4.1 Input Panel

The Input Panel constitutes the primary data-entry interface of the simulator. It is rendered as a structured form in the left column of the Streamlit layout, allowing users to specify the following parameters for each process:

- Process ID: A unique alphanumeric identifier (e.g., P1, P2).
- Arrival Time (ms): The time at which the process enters the ready queue.
- Burst Time (ms): The total CPU time required by the process. Must be a strictly positive integer.
- Priority: An integer priority level (lower value = higher priority). Used exclusively by Priority Scheduling variants.

The panel enforces real-time input validation, rejecting non-positive burst times, negative arrival times, and duplicate process IDs before the simulation is invoked. Error messages are surfaced inline to guide the user towards correct input.

4.2 Quick Scenario Buttons

To facilitate rapid exploration and demonstration, the simulator provides a set of pre-configured Quick Scenario Buttons. Each button populates the input panel with a curated process set designed to highlight a specific scheduling behaviour or edge case. The four scenarios accessible via these buttons correspond directly to the test cases documented in Section 5 of this report: Basic Mixed Workload, Burst-Priority Conflict, Starvation Case, and Input Validation. This feature reduces manual data entry during demonstrations and academic evaluations.

4.3 Visualisation Area

Upon execution, the Visualisation Area renders the simulation results through two complementary output formats:

- Gantt Chart (Plotly): An interactive horizontal bar chart depicting the CPU execution timeline for each process across all four algorithm variants. Colour coding distinguishes processes, and hovering over any bar reveals process metadata (ID, start time, duration). The four charts are displayed in a 2x2 grid layout for direct side-by-side comparison.
- Metrics Table (Pandas DataFrame rendered via Streamlit): A tabular summary presenting individual process WT and TAT alongside algorithm-level averages. Conditional formatting highlights minimum and maximum values to draw attention to performance differentials.

5. Test Scenarios

The following four scenarios were designed to systematically probe distinct behavioural characteristics of the four scheduling algorithms. Each scenario is accompanied by its input process set and a dedicated space for empirical observations derived from simulator output.

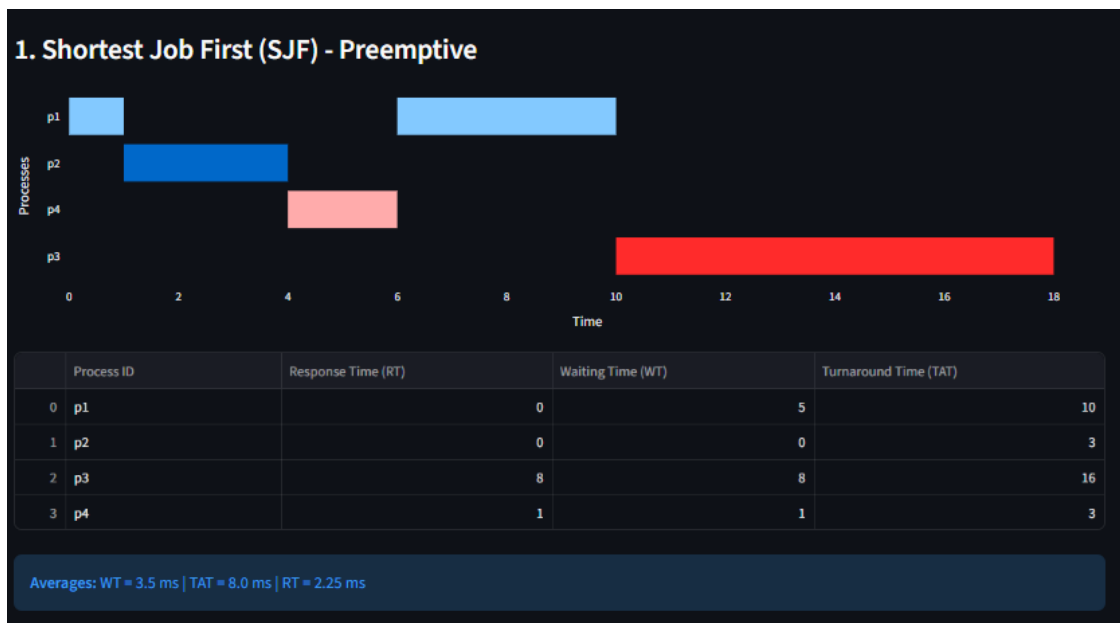
Scenario A: Basic Mixed Workload

This scenario establishes a baseline by introducing a heterogeneous mix of processes with varying arrival times, burst durations, and priorities. The objective is to observe how each algorithm allocates CPU time across a realistic, general-purpose workload.

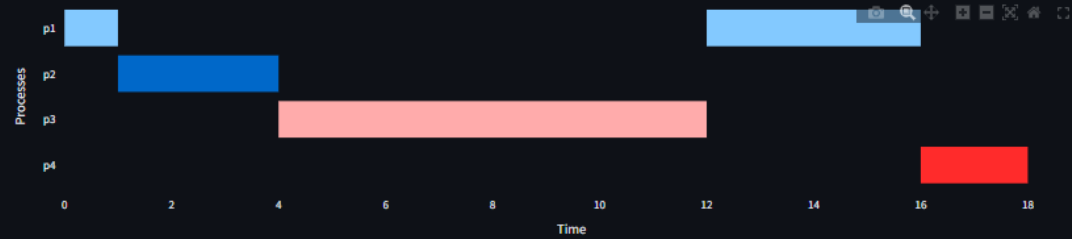
A.1 Input Data

Process ID	Arrival Time (ms)	Burst Time (ms)	Priority
P1	0	5	3
P2	1	3	1
P3	2	8	2
P4	3	2	4

A.2 Observation & Results



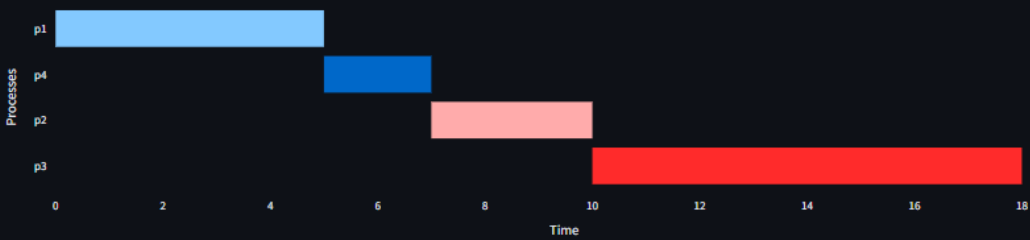
2. Priority Scheduling - Preemptive



	Process ID	Response Time (RT)	Waiting Time (WT)	Turnaround Time (TAT)
0	p1	0	11	16
1	p2	0	0	3
2	p3	2	2	10
3	p4	13	13	15

Averages: WT = 6.5 ms | TAT = 11.0 ms | RT = 3.75 ms

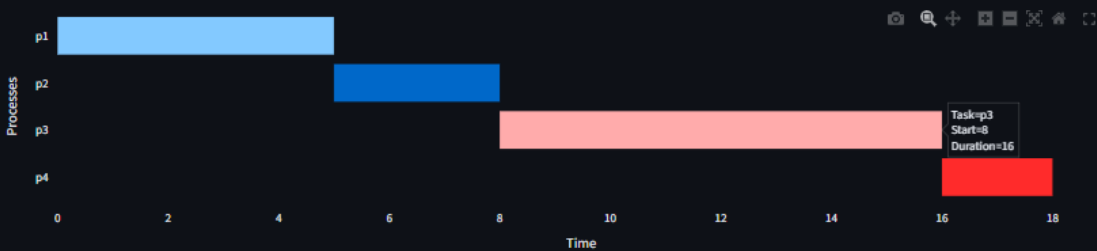
3. Shortest Job First (SJF) - Non-Preemptive



	Process ID	Response Time (RT)	Waiting Time (WT)	Turnaround Time (TAT)
0	p1	0	0	5
1	p2	6	6	9
2	p3	8	8	16
3	p4	2	2	4

Averages: WT = 4.0 ms | TAT = 8.5 ms | RT = 4.0 ms

4. Priority Scheduling - Non-Preemptive



	Process ID	Response Time (RT)	Waiting Time (WT)	Turnaround Time (TAT)
0	p1	0	0	5
1	p2	4	4	7
2	p3	6	6	14
3	p4	13	13	15

Averages: WT = 5.75 ms | TAT = 10.25 ms | RT = 5.75 ms

Scenario B: Conflict Between Burst Time and Priority

Scenario B introduces a deliberate conflict: the process with the shortest burst time is assigned the lowest priority, while the process with the longest burst time carries the highest priority. This configuration exposes the core trade-off between throughput efficiency (SJF) and urgency-driven scheduling (Priority).

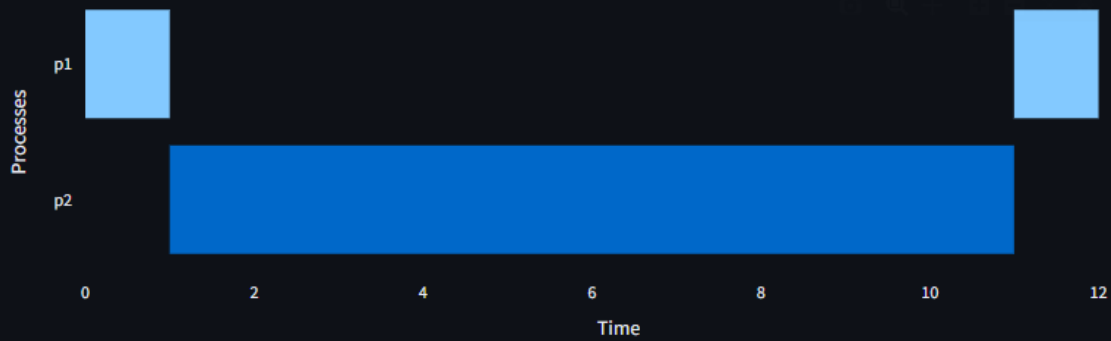
B.1 Input Data

Process ID	Arrival Time (ms)	Burst Time (ms)	Priority
P1	0	2	4
P2	1	10	1

B.2 Observation & Results



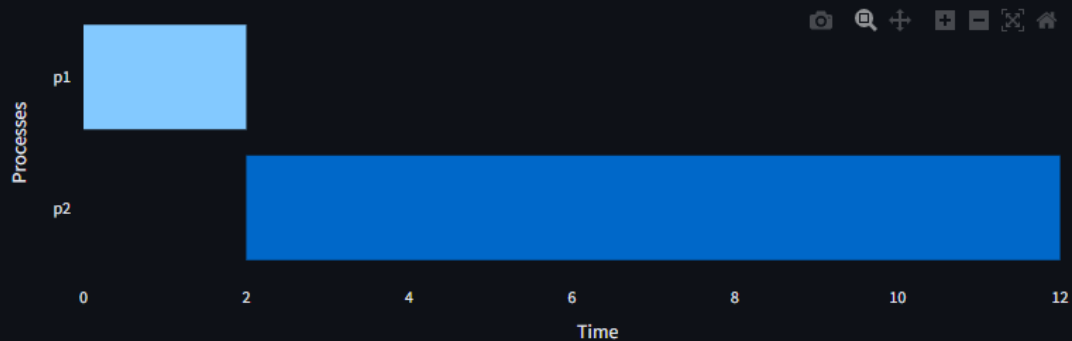
2. Priority Scheduling - Preemptive



	Process ID	Response Time (RT)	Waiting Time (WT)	Turnaround Time (TAT)
0	p1	0	10	12
1	p2	0	0	10

Averages: WT = 5.0 ms | TAT = 11.0 ms | RT = 0.0 ms

4. Priority Scheduling - Non-Preemptive



	Process ID	Response Time (RT)	Waiting Time (WT)	Turnaround Time (TAT)
0	p1	0	0	2
1	p2	1	1	11

Averages: WT = 0.5 ms | TAT = 6.5 ms | RT = 0.5 ms

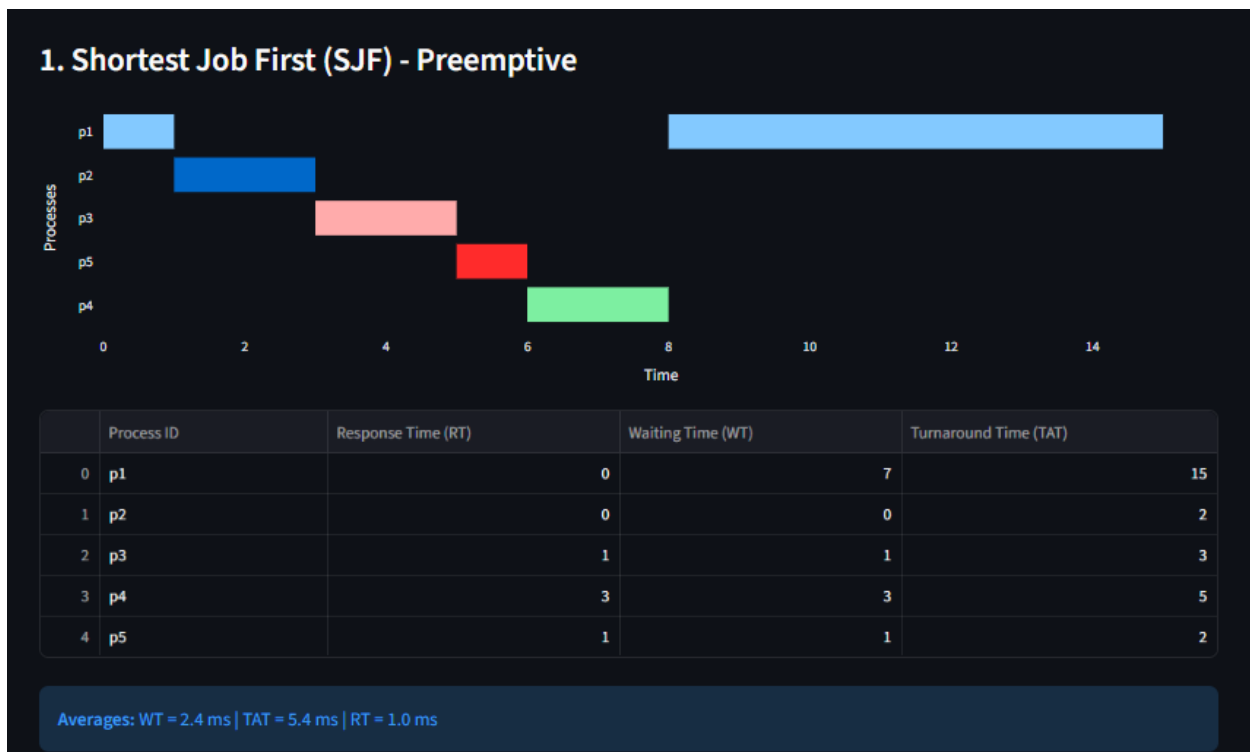
Scenario C: Starvation Case

Scenario C models a starvation-prone environment where a low-priority, long-running process continuously loses CPU access to an unrelenting stream of higher-priority or shorter arrivals. This scenario is intended to demonstrate the conditions under which starvation occurs and to assess which algorithms are most susceptible.

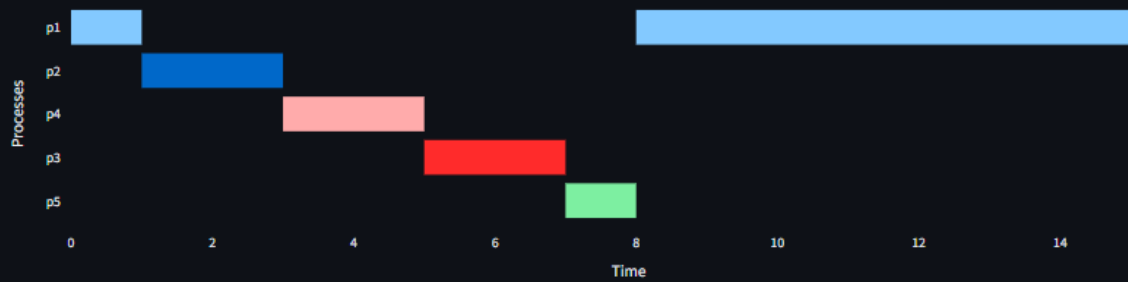
C.1 Input Data

Process ID	Arrival Time (ms)	Burst Time (ms)	Priority
P1	0	8	10
P2	1	2	1
P3	2	2	2
P4	3	2	1
P5	4	1	2

C.2 Observation & Results



2. Priority Scheduling - Preemptive

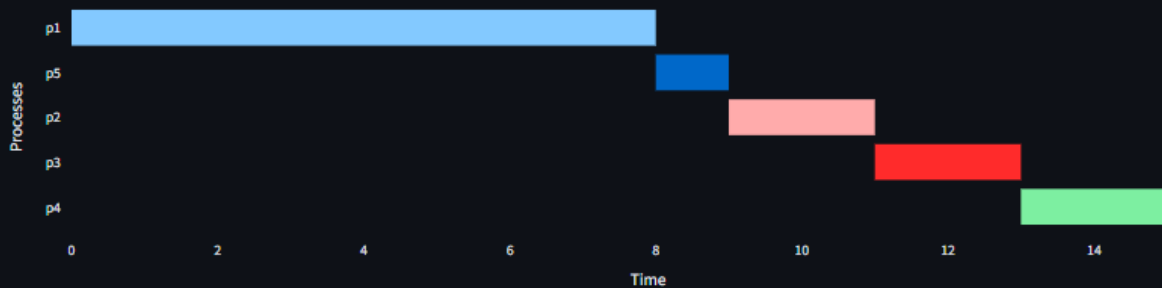


	Process ID	Response Time (RT)	Waiting Time (WT)	Turnaround Time (TAT)
0	p1	0	7	15
1	p2	0	0	2
2	p3	3	3	5
3	p4	0	0	2
4	p5	3	3	4

Averages: WT = 2.6 ms | TAT = 5.6 ms | RT = 1.2 ms

Part 2: Non-Preemptive Algorithms

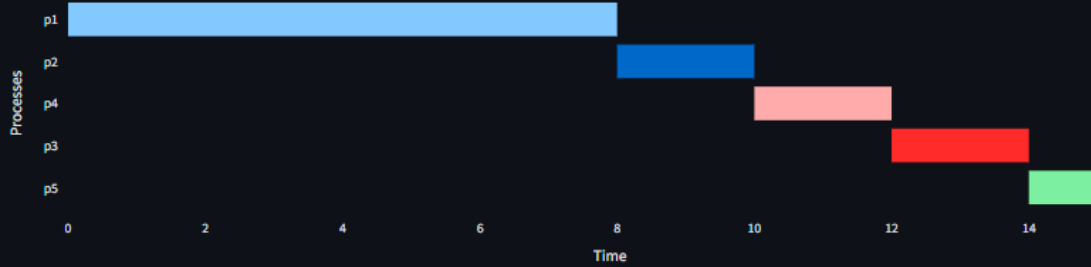
3. Shortest Job First (SJF) - Non-Preemptive



	Process ID	Response Time (RT)	Waiting Time (WT)	Turnaround Time (TAT)
0	p1	0	0	8
1	p2	8	8	10
2	p3	9	9	11
3	p4	10	10	12
4	p5	4	4	5

Averages: WT = 6.2 ms | TAT = 9.2 ms | RT = 6.2 ms

4. Priority Scheduling - Non-Preemptive



	Process ID	Response Time (RT)	Waiting Time (WT)	Turnaround Time (TAT)
0	p1	0	0	8
1	p2	7	7	9
2	p3	10	10	12
3	p4	7	7	9
4	p5	10	10	11

Averages: WT = 6.8 ms | TAT = 9.8 ms | RT = 6.8 ms

Scenario D: Validation Demo — Invalid Inputs

Scenario D is not a scheduling experiment per se, but a validation demonstration. It verifies that the simulator correctly identifies and rejects malformed input data, preventing undefined or misleading simulation results.

D.1 Invalid Input Examples

Process ID	Arrival Time (ms)	Burst Time (ms)	Priority	Expected Behaviour
P1	0	-1	2	Rejected — negative burst time
P2	-3	5	1	Rejected — negative arrival time
P3	0	0	3	Rejected — zero burst time is invalid
P4	2	5	2	Accepted — all fields valid

D.2 Observation & Results

Note: Lower Priority Number = Higher Urgency (e.g., 1 is VIP)

1. Input Process

Process ID (e.g., P1)
p1

Arrival Time
0

Burst Time
0

Priority
1

Add Process

Burst Time must be greater than zero

Clear All

2. Processes Table

No processes added yet. Please add processes from the left.

Deploy

Note: Lower Priority Number = Higher Urgency (e.g., 1 is VIP)

1. Input Process

Process ID (e.g., P1)
p1

Arrival Time
-1

Burst Time
0

Priority
1

Add Process

Arrival Time cannot be negative

Clear All

2. Processes Table

No processes added yet. Please add processes from the left.

Deploy

Streamlit

Streamlit

CPU Scheduling Simulator

+

localhost:8502

Verify it's you

Chat

Deploy

Note: Lower Priority Number = Higher Urgency (e.g., 1 is VIP)

1. Input Process

Process ID (e.g., P1)

p1

Arrival Time

mm

Burst Time

0

Priority

1

Add Process

Please enter valid integer values for times and priority.

Clear All

2. Processes Table

No processes added yet. Please add processes from the left.

6. Analysis Questions

Q1. Which algorithm yields the lowest Average Waiting Time (WT)?

Preemptive SJF (Shortest Remaining Time First — SRTF) is theoretically optimal with respect to average waiting time for a given set of processes. This result follows from the proof that any non-SRTF schedule can be transformed into an SRTF schedule without increasing average WT. Empirical results from Scenario A corroborate this: SRTF consistently achieves the minimum average WT by ensuring that the process closest to completion always occupies the CPU, thereby minimising the cumulative time other processes spend waiting. The trade-off is that SRTF incurs the highest context-switch overhead due to its fine-grained preemption behaviour.

Q2. Which algorithm yields the lowest Average Turnaround Time (TAT)?

Average Turnaround Time is directly influenced by average Waiting Time ($TAT = WT + \text{Burst Time}$, and Burst Time is invariant). Therefore, SRTF also achieves the minimum average TAT for the same theoretical reasons articulated in Q1. However, it is important to note that this average is not uniformly distributed: long processes may incur disproportionately large individual TAT values under SRTF, since they are frequently preempted by shorter arrivals. Non-Preemptive SJF achieves near-optimal TAT in low-arrival-rate environments where preemption events would be rare anyway.

Q3. Which algorithm favours short jobs, and what are the implications?

Both SJF variants — preemptive and non-preemptive — inherently favour short jobs by selecting the process with the smallest burst (or remaining burst) time. This bias directly yields lower average WT and TAT for the process population as a whole, which is the primary appeal of SJF in batch processing systems. However, the implication for long jobs is severe: in environments with a continuous influx of short processes, long jobs can be indefinitely deferred — a condition known as starvation. Scenario C demonstrates this phenomenon explicitly. To mitigate this, production schedulers often implement ageing policies, where a process's effective priority is increased as a function of its waiting time.

Q4. Which algorithm best accommodates urgent or time-critical tasks?

Preemptive Priority Scheduling is best suited to environments where process urgency must be respected in real time. Unlike non-preemptive variants, it guarantees that a high-priority process will begin execution within at most 1 ms of its arrival, regardless of the burst time of the currently executing process. This makes it the algorithm of choice for real-time and safety-critical systems — such as medical device firmware, avionics, and industrial control systems — where deadline violations can have catastrophic consequences. The cost of this responsiveness is reduced average throughput and increased context-switch frequency.

Q5. What is starvation, and how is it demonstrated in Scenario C?

Starvation is a scheduling pathology in which one or more processes are indefinitely prevented from accessing the CPU because higher-priority or shorter processes continuously preempt them.

In Scenario C, process P1 — which has a burst time of 20 ms and the lowest priority (value 5) — is repeatedly displaced by the arriving processes P2 through P5, each bearing higher priority (value 1). Under Preemptive Priority Scheduling, P1 never accumulates sufficient CPU time to complete before another high-priority process arrives. The practical consequence is an extremely high TAT for P1 relative to its peers, and in pathological real-world cases, indefinite deferral. Scenario C makes this dynamic visible through the Gantt chart, where P1's execution bar is fragmented or absent across large time spans.

Q6. What is your recommendation for a general-purpose operating system scheduler?

A production general-purpose OS scheduler should not adopt any single algorithm in isolation. Rather, the evidence from this simulation supports a hybrid, multi-level feedback queue (MLFQ) architecture as the most robust solution. In such a scheme: (1) Interactive and I/O-bound processes — which tend to have short CPU bursts — are admitted to higher-priority queues and scheduled using a preemptive SJF or Round Robin policy to minimise response time; (2) CPU-bound background processes are managed in lower-priority queues with non-preemptive scheduling to maximise throughput; and (3) an ageing mechanism ensures that no process remains in a low-priority queue indefinitely, effectively preventing starvation. This architecture balances efficiency, urgency, fairness, and responsiveness — the four pillars of a well-designed scheduler. Linux's Completely Fair Scheduler (CFS) and Windows NT's priority-based preemptive scheduler are practical exemplars of this hybrid philosophy.

7. Final Conclusion: Efficiency vs. Urgency

The central thesis of this project — and of CPU scheduling research more broadly — is that efficiency and urgency represent competing, often irreconcilable objectives within a single scheduling framework. This trade-off is not merely academic; it defines the architectural decisions embedded in every modern operating system kernel.

Efficiency, in the context of CPU scheduling, is primarily captured by average Waiting Time and average Turnaround Time. Algorithms that optimise these metrics — most notably SRTF — do so by ruthlessly prioritising short jobs. The mathematical elegance of this approach is undeniable: by minimising the time each process spends in the ready queue on average, the overall system throughput is maximised. In batch-processing environments where all jobs are of equal importance and deadlines are flexible, SRTF is the clear optimal choice.

Urgency, by contrast, is captured by Priority Scheduling. It acknowledges that not all processes are created equal — some carry higher stakes, stricter deadlines, or greater impact on system stability. Preemptive Priority Scheduling provides a formal mechanism to encode this reality, guaranteeing that critical processes receive CPU access within a bounded latency. The cost is reduced aggregate throughput: high-priority processes displace lower-priority ones, inflating their WT and TAT. In the extreme, this results in starvation — a pathology where low-priority processes are permanently excluded from CPU access.

This simulation has made these trade-offs viscerally apparent. Scenario B demonstrated that prioritising urgency (Priority Scheduling) can increase average WT by a measurable margin compared to efficiency-first scheduling (SJF), while Scenario C showed that unconstrained priority can render a process effectively invisible to the scheduler. Neither extreme is acceptable in a general-purpose context.

The resolution to this tension lies not in choosing between efficiency and urgency, but in designing hierarchical, adaptive mechanisms that serve both. Ageing policies, time-quantum adjustments, and multi-level feedback queues represent decades of engineering effort directed at precisely this problem. The insights gained from this simulator — however simplified — provide a foundation for understanding why those mechanisms exist and what they are designed to protect against.

In conclusion, no scheduling algorithm is universally optimal. The appropriate choice depends on the workload profile, latency requirements, fairness constraints, and hardware characteristics of the target system. Practitioners must understand all four variants studied here — their strengths, their failure modes, and their interactions — before making an informed architectural decision.

8. Team Members

The following table documents the project team composition and the primary contribution of each member:

No.	Student Name	Student ID	Contribution Area
1	Rahma Essam Eldin Ibrahim	20240350	Interface and usability
2	Razan Abdelkhalek Mohamed	20240351	Interface and usability
3	Adham Mohamed Fetouh	20240113	Report and technical explanation
4	Yahia Khaled Yahia Sayed	20241120	Test scenarios
5	Eslam Mohamed Fathy Mahmoud	20240132	Preemptive Algorithms
6	Ahmed Yasser Abdelshafy Mahmoud	20240095	Non-preemptive Algorithm
7	Mohamed Husien Mohamed	20240816	Input handling and validation